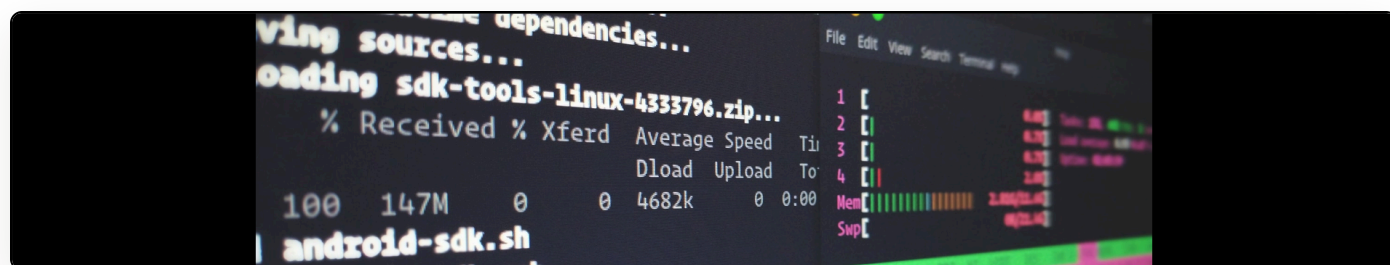


Writing a plugin for Open Journal Systems: Part 1

Cite as: Martin Paul Eve, "Writing a plugin for Open Journal Systems: Part 1", *eve.gd: Martin Paul Eve*, September 17, 2011, <https://doi.org/10.59348/ez7p0-cvj16>.



As promised when I described [the problem I was having](#) with the xmlGalley plugin in OJS, I'm going to begin describing the path I am taking to fixing this, and hope that the knowledge will provide some shortcuts for others wishing to develop plugins, amid the sparse documentation on this aspect. As OJS is the leading system for Open Access publishing, it is important that technical resources are available for others to build upon this platform.

The structure of an OJS plugin

This tutorial will be covering the basics of writing a "generic" plugin. These live under `plugins/generic/pluginName` and should contain, at a minimum, the following files:

`index.php`

`version.xml`

The `index.php` file is a simple loader. It should contain code that instantiates your main plugin file and returns a new instance of that plugin. In my project, it looks like this:

```

<?php

/**
 * @defgroup plugins_generic_meXml
 */

/**
 * @file plugins/generic/meXml/index.php
 *
 * Copyright (c) 2011 Martin Paul Eve
 * Distributed under the GNU GPL v2. For full terms see the file docs/COPYING.
 *
 * @ingroup plugins_generic_meXml
 * @brief Wrapper for meXml
 *
 */

// $Id$

require_once('meXml.inc.php');

return new meXml();

?>

```

The version.xml file is a file used by the internals of the OJS plugin installer. It specifies the version so that, when OJS goes to install the plugin, it knows whether to upgrade or not. **The version.xml file is important** because, if you just put your files into the OJS directory, rather than going through the proper installer, there will not be a corresponding line in the plugins table of the database and **you will not be able to enable/disable your plugin**. The "application" is the plugin name and "class" is the main class of the plugin, as specified in the index.php file. My version.xml file looks like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE version SYSTEM "../.../lib/pkp/dtd/pluginVersion.dtd">
<version>
  <application>meXml</application>
  <type>plugins.generic</type>
  <release>1.0.0.0</release>
  <date>2011-09-11</date>
  <lazy-load>1</lazy-load>
  <class>meXml</class>
</version>
```

Starting out

So now you've got a basic structure, you'll need to create the file that index.php is loading, in my case called meXml.php. This needs to be fleshed out with some initial functions: register, getName, getDisplayName, getDescription, getManagementVerbs, setEnabled and manage. Optionally, you can include a getInstallSchemaFile function. Here's a stub:

```
<?php
```

```
/**
 * @file meXml.inc.php
 *
 * Copyright (c) 2011 Martin Paul Eve
 * Distributed under the GNU GPL v2. For full terms see the file docs/COPYING.
 *
 * Portions copyright (c) 2003-2011 John Willinsky
 *
 * @class meXml
 * @ingroup plugins_generic_meXml
 *
 * @brief Martin Eve's modified XML Galley Plugin
 */

import('lib.pkp.classes.plugins.GenericPlugin');

class meXml extends GenericPlugin {
    function register($category, $path) {
        if (parent::register($category, $path)) {
            if ($this->getEnabled()) {
                HookRegistry::register('ArticleGalleyDAO::insertNewGalley', array(&$this, 'callback')
                );

                return true;
            }
            return false;
        }

        function callback($hookName, $args) {
            // code here
        }

        function getName() {
            return 'meXml';
        }

        function getDisplayName() {
            return "Martin Eve's modified XML Galley Plugin";
        }

        function getDescription() {
            return "Allows PDF galleys to be generated from XML";
        }
    }
}
```

```

/**
 * Get the filename of the ADODB schema for this plugin.
 */
function getInstallSchemaFile() {
    return $this->getPluginPath() . '/' . 'schema.xml';
}

/**
 * Set the enabled/disabled state of this plugin
 */
function setEnabled($enabled) {
    parent::setEnabled($enabled);
    $journal =& Request::getJournal();
    return false;
}

/**
 * Display verbs for the management interface.
 */
function getManagementVerbs() {
    $verbs = array();
    if ($this->getEnabled()) {
        $verbs[] = array('settings',
Locale::translate('plugins.generic.xmlGalley.manager.settings'));
    }
    return parent::getManagementVerbs($verbs);
}

/*
 * Execute a management verb on this plugin
 * @param $verb string
 * @param $args array
 * @param $message string Location for the plugin to put a result msg
 * @return boolean
 */
function manage($verb, $args, &$message) {
    if (!parent::manage($verb, $args, $message)) return false;

    $journal =& Request::getJournal();

    $templateMgr =& TemplateManager::getManager();
    $templateMgr->register_function('plugin_url', array(&$this,
'smartyPluginUrl'));

    $this->import('XMLGalleySettingsForm');
    $form = new XMLGalleySettingsForm($this, $journal->getId());

```

```

        switch ($verb) {
            case 'settings':

Locale::requireComponents(array(LOCALE_COMPONENT_APPLICATION_COMMON,
LOCALE_COMPONENT_PKP_MANAGER));
                                // if we are updating XSLT settings or switching XSL
                                sheets

                                if (Request::getUserVar('save')) {
                                    $form->readInputData();
                                    $form->initData();
                                    if ($form->validate()) {
                                        $form->execute();
                                    }
                                    $form->display();
                                }
                                return true;
            default:
                                // Unknown management verb
                                assert(false);
                                return false;
        }
    }
}
?>

```

This basic stub has all the requisite functions for a plugin. The **register** function is called when OJS initializes a page and, in this case, the plugin specifies that it wants a hook into ArticleGalleyDAO::insertNewGalley, to the function "callback", within itself (&\$this). For a more comprehensive overview of how the manage function is working, to here display a tpl form, I'd suggest viewing the original xmlGalley source, from which this is derived.

The only remaining file to explain is the schema.xml file. This is, again, executed when OJS installs the plugin and is used to create the necessary database tables. For xmlGalley, it looks like this:

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE schema SYSTEM "../.../lib/pkp/dtd/xmlSchema.dtd">

<!--
* schema.xml
*
* Copyright (c) 2003-2011 John Willinsky
* Distributed under the GNU GPL v2. For full terms see the file docs/COPYING.
*
* XML Galley plugin schema in XML.
*
* $Id$
-->

<schema version="0.2">

    <!--
    *
    * TABLE article_xml_galleys
    *
    -->
    <table name="article_xml_galleys">
        <field name="xml_galley_id" type="I8">
            <KEY/>
            <AUTOINCREMENT/>
        </field>
        <field name="galley_id" type="I8">
            <NOTNULL/>
        </field>
        <field name="article_id" type="I8">
            <NOTNULL/>
        </field>
        <field name="label" type="C2" size="32">
            <NOTNULL/>
        </field>
        <field name="galley_type" type="C2" size="255">
            <NOTNULL/>
        </field>
        <field name="views" type="I4">
            <NOTNULL/>
            <DEFAULT VALUE="0"/>
        </field>
        <descr>XML-Derived galleys.</descr>
    </table>

</schema>

```

Testing

To test the stub plugin, you'll need to compress the structure into a .tar.gz file. On GNU/Linux systems, you can achieve this with: `tar -pczf name_of_archive.tar.gz /path/to/directory`

After this, run the plugin through the install procedure and you should be up and running.